

Министерство науки и высшего образования Российской Федерации Челябинский
государственный университет

**Автоматизированная MLOps-система
детектирования дрифта данных и
адаптивного переобучения моделей
машинного обучения**

Аннотация

In machine learning operations systems, machine learning models are often exposed to vulnerabilities one of the most prominent being data drift. Data drift can be defined as the variance in statistical distribution of input data differing from what was used during model training, thus leading to degradation in performance of the machine learning model over time. In the midst of the Artificial Intelligence revolution, machine learning systems become more and more critical in production environments, models must maintain accuracy despite changes in data, this entailing the need for automated solutions capable of detecting anomalies and initiating corrective actions. Existing solutions for model monitoring treat drift detection, model management, and retraining as separate concerns, requiring the combination of multiple tools. To address this gap, this thesis proposes the development of a unified machine learning operations system monitoring tool that automatically monitors production data for distributional offsets, evaluates their severity using statistical methods and triggers the retraining of the model when a critical drift is detected. The system combines drift detection algorithms, workflow orchestration and automated retraining pipelines in a unified platform, eliminating the need for manual intervention while ensuring the continuous reliability of the model.

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ

1. ИССЛЕДОВАНИЕ ПРЕДМЕТНОЙ ОБЛАСТИ

- 1.1. Проблематика деградации моделей машинного обучения в производственных средах
- 1.2. Анализ современных подходов к мониторингу распределений данных
- 1.3. Сопоставление платформ и инструментов для отслеживания качества моделей
- 1.4. Методологические основы статистического анализа смещений распределений
- 1.5. Оценка эффективности различных статистических критериев для выявления дрифта
- 1.6. Современные практики автоматизации процессов обновления моделей
- 1.7. Характеристика экспериментальных данных и источников их получения
- 1.8. Предлагаемый подход к построению единой платформы мониторинга и адаптации моделей

2. РАЗРАБОТКА АРХИТЕКТУРЫ СИСТЕМЫ

- 2.1. Функциональные и нефункциональные требования
- 2.2. Общая архитектура интегрированной платформы
- 2.3. Проектирование компонента выявления дрифта
- 2.4. Проектирование компонента управления рабочими процессами
- 2.5. Проектирование компонента автоматизированного переобучения
- 2.6. Проектирование компонента хранения и версионирования артефактов
- 2.7. Схемы интеграции и взаимодействия модулей

3. РАЗРАБОТКА И ВНЕДРЕНИЕ СИСТЕМЫ

- 3.1. Выбор технологического стека
- 3.2. Организация кодовой базы и модульная структура
- 3.3. Разработка механизма детектирования дрифта

3.4. Разработка механизма оркестрации

3.5. Разработка механизма автоматического переобучения

3.6. Разработка механизма управления жизненным циклом моделей

3.7. Разработка программных интерфейсов и панели мониторинга

3.8. Демонстрация функциональности системы

4. ЭКСПЕРИМЕНТАЛЬНАЯ ВАЛИДАЦИЯ

4.1. Стратегия проведения экспериментов

4.2. Валидация компонента детектирования дрифта

4.3. Валидация компонента автоматического переобучения

4.4. Комплексное тестирование системы

4.5. Практическая апробация на реальных наборах данных

ЗАКЛЮЧЕНИЕ

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

ПРИЛОЖЕНИЯ

ВВЕДЕНИЕ

Актуальность

The deployment of machine learning models in production has accelerated over the past decade. Early deployments assumed static data distributions, but real-world data changes over time. This mismatch between training and production distributions is a fundamental challenge in production ML.

The problem gained attention with Sculley et al. (2015), which highlighted technical debt in ML systems, including data distribution shifts. Since then, research has focused on detecting and adapting to these changes.

Theoretical Foundations: Understanding Drift in Machine Learning In supervised learning, a model learns a mapping $f: X \rightarrow Y$ from training data (X_{train}, Y_{train}) . The model assumes that production data (X_{prod}, Y_{prod}) follows the same distribution as training data. When this assumption fails, model performance degrades. Formally, distribution shift occurs when:

$$P_{train}(X, Y) \neq P_{prod}(X, Y)$$

This inequality can manifest in different ways, leading to distinct types of drift.

Taxonomy of Drift Types Research categorizes drift into several types.

Data Drift (Covariate Shift) occurs when the distribution of input features $P(X)$ changes while the conditional distribution $P(Y|X)$ remains constant. Mathematically, $P_{train}(X) \neq P_{prod}(X)$ while $P_{train}(Y|X) = P_{prod}(Y|X)$. Common examples include shifts in customer demographics, sensor calibration changes, modifications to feature engineering pipelines, and seasonal variations in input patterns. These changes affect the input space without altering the relationship between features and targets.

Concept Drift (Label Shift) occurs when the relationship between inputs and outputs changes, while input distributions may remain stable. $P_{train}(Y|X) \neq P_{prod}(Y|X)$. Examples include evolving fraud patterns where the same transaction characteristics indicate fraud differently over time, changing customer preferences that alter the relationship between user features and purchase behaviour, and market dynamics that modify how economic indicators relate to outcomes.

Prior Probability Shift involves changes in the marginal distribution of target variables, where $P_{train}(Y) \neq P_{prod}(Y)$. This occurs when the overall prevalence of classes or target values changes, such as when the proportion of fraudulent transactions increases over time, even if the relationship between features and fraud remains constant.

Subspace Drift occurs when distribution changes affect only a subset of features while others remain stable. This partial drift can be particularly challenging to detect, as the overall feature space may appear stable while critical features experience significant shifts.

Data drift is the most common and detectable form of drift in production systems. Input distributions change more frequently than concept relationships and data drift often precedes or accompanies concept drift. Early detection of data drift can prevent downstream performance degradation. A key

advantage of data drift detection is that it can be performed using only input features XX , without requiring labelled production data. Concept drift detection typically requires labelled production data (X, Y) (X, Y) , which is often unavailable or delayed in real-world scenarios. This makes data drift detection practical for real-time monitoring systems where labels may not be immediately available. The statistical methods for detecting data drift are well established and interpretable. Tests such as Population Stability Index (PSI), Kolmogorov-Smirnov test, and Chi-square test directly measure distribution differences and provide actionable metrics. These methods allow for setting thresholds based on both statistical significance and business impact. From an operational perspective, data drift often indicates upstream issues in data pipelines, changes in external factors, or shifts in the underlying population. Early detection enables proactive intervention before model performance degrades significantly. Monitoring input distributions is computationally efficient and scalable; making it feasible to track thousands of features simultaneously in high-volume production systems. Data drift detection serves as the foundation for adaptive systems. It provides the trigger mechanism for automated model updates and informs retraining strategies by identifying which features have changed and to what extent. Understanding input distribution changes helps determine whether incremental updates, full retraining, or feature engineering modifications are necessary.

The fragmentation of existing solutions creates significant operational challenges. Combining these tools requires extensive custom integration code connecting multiple systems, manual configuration of thresholds and triggers across different platforms, complex deployment architectures with multiple components to maintain, and ongoing maintenance of integrations as tools evolve. This fragmentation increases time-to-detection and time-to-resolution when drift occurs, as information must flow through multiple systems and manual steps. Operational complexity grows with the number of integrated tools, increasing the maintenance burden and the risk of configuration errors. The lack of unified visibility makes it difficult to understand the complete state of model health and drift conditions across an organization's ML systems.

An integrated MLOps system that unifies automated drift detection using statistical methods, intelligent orchestration that manages workflows and dependencies, automated retraining triggered by drift severity, and model lifecycle management with versioning and deployment would address these challenges comprehensively. Such a system would reduce operational overhead through automation, eliminating the need for manual coordination between multiple tools. Response time to distribution changes would improve significantly, as detection and retraining would occur in a single, optimized workflow. Continuous model reliability would be ensured through systematic monitoring and adaptation. A single platform for MLOps workflows would simplify deployment, reduce training requirements, and provide unified visibility into model health. Most importantly, such a system would demonstrate the feasibility and value of unified MLOps architectures, advancing the state of the art in production machine learning operations.

Problem Statement

1. The core problem is the absence of a unified, open-source system that:
2. Automatically monitors production data for distributional shifts
3. Evaluates drift severity using statistical methods
4. Triggers automated retraining when critical drift is detected
5. Manages the complete lifecycle from detection to deployment

Existing solutions treat these concerns separately, requiring multiple tools and manual coordination. This fragmentation increases complexity, maintenance overhead, and the risk of delayed responses to drift.

Research Objectives

The primary objective is to design and implement an integrated MLOps system that automatically detects data drift and triggers adaptive retraining of machine learning models.

Specific objectives include:

1. Develop a unified architecture combining drift detection, orchestration, and retraining
2. Implement statistical methods (PSI, Kolmogorov-Smirnov, Chi-square) for drift detection
3. Design an orchestration mechanism for automated workflow execution
4. Create an automated retraining pipeline that collects data, merges datasets, trains models, evaluates performance, and deploys improved models
5. Validate the system on real-world datasets with known drift patterns
6. Research Tasks
7. To achieve these objectives, the following tasks are defined:
8. Analyze existing solutions for drift detection, model monitoring, and automated retraining
9. Review and compare statistical methods for detecting distributional shifts
10. Design the architecture of an integrated system with clearly defined components and interactions
11. Implement drift detection algorithms from fundamental libraries (Numpy, scipy, pandas)
12. Develop a custom orchestration system for managing workflows and dependencies
13. Build a model registry and data versioning system for tracking model and data lineage
14. Implement an automated retraining pipeline with evaluation and deployment logic
15. Design and implement APIs and monitoring interfaces for system interaction
16. Conduct experimental validation on synthetic and real-world datasets
17. Evaluate system performance, detection accuracy, and retraining effectiveness

1.1. Проблематика деградации моделей машинного обучения в производственных средах

Model degradation in production is a fundamental issue affecting deployed ML systems. Models trained on historical data face distribution shifts as conditions change, leading to declining performance. The problem stems from the assumption that production data follows the same distribution as training data. In practice, this rarely holds over time. Performance may degrade gradually or suddenly, depending on the nature of the shift. **Mechanisms of**

Performance Degradation

Degradation can occur through several mechanisms. Covariate shift changes input feature distributions while preserving the conditional relationship between inputs and outputs. As production inputs diverge from training, the model encounters unseen regions of the feature space, reducing accuracy and reliability. Concept drift changes the relationship between inputs and outputs. The model's learned mapping becomes outdated, even when inputs remain similar. This is common in dynamic domains such as fraud detection, recommendation systems, and cybersecurity. Prior probability shift changes the marginal distribution of target variables. The model may be calibrated for one class distribution while production reflects another, causing systematic errors.

Temporal Aspects of Degradation

Degradation patterns vary over time. Sudden drift can cause immediate performance drops, often due to external events, policy changes, or data pipeline modifications. Gradual drift leads to slow decline, making it harder to detect but still impactful. Incremental drift involves small, continuous changes that accumulate. Recurring drift follows periodic patterns, such as seasonal effects.

Consequences of Undetected Degradation

The consequences are significant. In financial services, fraud detection models may produce more false positives or miss new fraud patterns, affecting costs and customer experience. In recommendation systems, outdated models reduce relevance, engagement, and revenue. In healthcare diagnostics, degraded models risk patient safety. In autonomous systems, undetected drift can compromise safety and reliability.

Challenges in Production Environments

Production monitoring faces several challenges. Labeled production data is often unavailable or delayed, making concept drift detection difficult. Input drift can be detected without labels, but evaluation still requires labels. Modern systems process large volumes of data with high-dimensional feature spaces, making manual monitoring impractical. Computational efficiency matters for real-time or near-real-time systems. Detection must be fast enough to support

timely responses. False positives can trigger unnecessary retraining; false negatives allow degraded performance to persist.

The Imperative for Automated Monitoring

Given these challenges, automated monitoring and adaptive retraining are necessary. Continuous monitoring must detect distribution shifts early, quantify severity, and trigger appropriate responses. Without automation, performance degradation can persist, causing financial losses, safety risks, and poor user experience.

1.2. Анализ современных подходов к мониторингу распределений данных

Modern approaches to monitoring data distributions fall into several categories based on methodology, application domain, and integration capabilities. These approaches address distribution shifts using statistical tests, distance metrics, and model-based methods.

Statistical Testing Approaches

Statistical testing compares production distributions to training baselines using hypothesis tests. Population Stability Index (PSI) quantifies distribution differences through bin-wise comparisons, widely used in financial services for its interpretability and thresholds. Kolmogorov-Smirnov tests compare empirical cumulative distribution functions, providing p-values for significance. Chi-square tests assess categorical feature shifts by comparing observed and expected frequencies. These methods are interpretable, computationally efficient, and well-established.

Distance-Based Methods

Distance-based methods measure the distance between training and production distributions. Wasserstein distance (Earth Mover's Distance) quantifies the cost of transforming one distribution into another, useful for continuous features. Maximum Mean Discrepancy (MMD) uses kernel methods to compare distributions in high-dimensional spaces. Kullback-Leibler divergence measures information loss when approximating one distribution with another. Total Variation Distance provides an upper bound on the difference between distributions.

Model-Based Detection Methods

Model-based methods leverage ML models to detect shifts. Two-sample tests use classifiers to distinguish between training and production data, with performance indicating drift. Performance-based detection monitors accuracy, precision, recall, or other metrics, detecting concept drift but requiring labels. Density ratio estimation models the ratio between training and production densities to identify shifts.

Streaming and Incremental Approaches

Streaming approaches handle continuous data flows. Adaptive windowing adjusts window sizes based on detected change points. Statistical process control tracks metrics over time to detect anomalies. Change point detection identifies times when distribution properties change significantly.

Feature-Level vs. Dataset-Level Monitoring

Feature-level monitoring assesses each feature individually, providing interpretability but requiring aggregation for dataset-level decisions. Dataset-level monitoring evaluates the entire feature space simultaneously, useful for high-dimensional data but less interpretable.

Hybrid and Ensemble Approaches

Hybrid methods combine multiple techniques to improve robustness. Ensemble detection aggregates results from multiple tests, reducing false positives and negatives. Multi-scale approaches apply methods at different granularities. Adaptive threshold selection adjusts sensitivity based on historical patterns.

Integration with MLOps Workflows

Approaches vary in their integration with MLOps. Standalone tools focus on detection and visualization. Integrated platforms combine detection with alerting, logging, and visualization. Automated systems connect detection to retraining and deployment workflows.

Limitations and Challenges

Limitations include the need for labeled data for some methods, computational complexity in high dimensions, threshold selection without clear guidelines, and handling of missing or corrupted data. Multi-dimensional drift detection remains challenging.

Emerging Trends

Recent trends include automated threshold optimization, explainability for drift detection results, and real-time detection for streaming systems. Active learning approaches reduce labeling requirements, and domain adaptation techniques enable model adaptation to shifted distributions.

1.3. Сопоставление платформ и инструментов для отслеживания качества моделей

Various platforms and tools address model quality tracking, each with different strengths and limitations. This section compares major platforms across detection capabilities, model management, orchestration, and integration features.

Evidently AI

Evidently AI is an open-source monitoring platform focused on data and model quality. It provides statistical tests including PSI, Kolmogorov-Smirnov, and Chi-square, generates reports and dashboards, supports batch and real-time monitoring, and includes data quality checks. However, it lacks built-in automated retraining and relies on external orchestration tools.

NannyML

NannyML specializes in post-deployment monitoring without requiring production labels. It includes performance estimation using confidence-based approaches, drift detection, and data quality monitoring. It excels at performance estimation but does not provide automated retraining or comprehensive orchestration.

MLflow

MLflow is a platform for managing the ML lifecycle. It includes experiment tracking, model registry with versioning, model serving capabilities, and some drift detection plugins. However, drift detection is not core, and retraining requires custom pipelines and external orchestration.

Weights & Biases (W&B)

W&B provides experiment tracking, model versioning, hyperparameter optimization, and some model monitoring. It focuses on experiment management rather than production monitoring, and drift detection requires custom implementation.

Alibi Detect

Alibi Detect is a library for outlier, adversarial, and drift detection. It offers multiple detection algorithms, including statistical tests and model-based methods, and is designed for integration into custom pipelines. However, it does not include model management, orchestration, or automated retraining.

Fiddler AI

Fiddler AI is a commercial platform with comprehensive monitoring, model explainability, and alerting. It requires licensing and is not open-source.

Aporia

Aporia provides model monitoring with drift detection and performance tracking. It is commercial and requires licensing.

Open Source Components

Several specialized tools address specific aspects. scikit-multiflow focuses on streaming ML and concept drift detection. River (creme) provides incremental learning and drift detection. Apache Airflow and Prefect handle workflow orchestration but do not include drift detection.

Comparison Matrix

Platforms vary across detection methods, model management, orchestration, retraining automation, open-source availability, ease of integration, and scalability. Evidently AI and NannyML excel at detection; MLflow and W&B excel at model management; Airflow and Prefect excel at orchestration. None provide a fully integrated solution combining all aspects.

Integration Challenges

Using multiple tools requires custom integration, configuration management across platforms, deployment complexity, and ongoing maintenance. This fragmentation increases operational overhead and the risk of missed drift events.

Gap Analysis

The primary gap is the lack of a unified, open-source platform that combines drift detection, workflow orchestration, automated retraining, and model management. Existing solutions excel in specific areas but require significant integration effort to achieve end-to-end automation.

1.4. Методологические основы статистического анализа смещений распределений

Statistical methods for detecting distribution shifts rely on comparing training and production distributions. This section covers the theoretical foundations of key methods. **Population Stability Index (PSI)** PSI quantifies distribution differences by comparing binned frequencies. For a feature with values divided into bins, PSI is computed as:

$$PSI = \sum_{i=1}^n (P_i - Q_i) \cdot \ln\left(\frac{P_i}{Q_i}\right)$$

Where P_i is the proportion of baseline data in bin i , and Q_i is the proportion of production data in bin i . Interpretation: $PSI < 0.1$ suggests no significant drift, $0.1-0.25$ indicates minor drift, and > 0.25 indicates major drift. PSI is symmetric, interpretable, and handles both continuous and categorical data. Limitations include sensitivity to binning and potential masking of local shifts.

Kolmogorov-Smirnov Test

The KS test compares two cumulative distribution functions. The test statistic is:

$$D_n = \sup_x |F_n(x) - F_0(x)|$$

Where $F_n(x)$ is the empirical CDF of the sample and $F_0(x)$ is the reference CDF. For two-sample comparison:

$$D_{n,m} = \sup_x |F_n(x) - G_m(x)|$$

Where F_n and G_m are empirical CDFs. The null hypothesis is that samples come from the same distribution. A small p-value indicates drift. Advantages include non-parametric nature and distribution-free properties. Limitations include lower power in tails and reduced effectiveness in high dimensions.

Chi-Square Test

The Chi-square test compares observed and expected frequencies in categorical data. The test statistic is:

$$\chi^2 = \sum_{i=1}^k \frac{(O_i - E_i)^2}{E_i}$$

Where O_i are observed frequencies and E_i are expected frequencies. Under the null hypothesis, χ^2 follows a Chi-square distribution with $(k-1)$ degrees of freedom. A large χ^2 or small p-value indicates drift. It applies to categorical features, is well established, and provides clear significance testing. Limitations include sensitivity to low expected frequencies and reduced effectiveness with many categories.

Wasserstein Distance (Earth Mover's Distance)

Wasserstein distance measures the cost of transforming one distribution into another. For one-dimensional distributions, the 1-Wasserstein distance is:

$$W_1(P, Q) = \int_{-\infty}^{\infty} |F_P(x) - F_Q(x)| dx$$

Where F_P and F_Q are CDFs. For empirical distributions, it can be computed using the difference of quantiles. Higher values indicate greater drift. Advantages include interpretability as transport cost, sensitivity to distribution shape, and applicability to continuous data. Limitations include computational complexity in high dimensions and less established thresholds.

Maximum Mean Discrepancy (MMD)

MMD uses kernel methods to compare distributions in reproducing kernel Hilbert space (RKHS). The squared MMD is:

$$MMD^2(P, Q) = E_{x, x' \sim P} [k(x, x')] - 2E_{x \sim P, y \sim Q} [k(x, y)] + E_{y, y' \sim Q} [k(y, y')]$$

Where $k(\cdot, \cdot)$ is a kernel function. For empirical distributions, the unbiased estimator is used. Larger MMD values indicate greater drift. Advantages include applicability to high-dimensional data, non-parametric nature, and kernel flexibility. Limitations include kernel selection sensitivity and computational cost.

Kullback-Leibler Divergence

KL divergence measures information loss when approximating one distribution with another:

$$D_{KL}(P \parallel Q) = \int_{-\infty}^{\infty} p(x) \ln \left(\frac{p(x)}{q(x)} \right) dx$$

For discrete distributions:

$$D_{KL}(P \parallel Q) = \sum_i p_i \ln \left(\frac{p_i}{q_i} \right)$$

KL divergence is non-negative and asymmetric. Higher values indicate greater divergence. It has strong information-theoretic foundations but is asymmetric and requires density estimation.

Total Variation Distance

Total variation distance is:

$$TV(P, Q) = \sup_A |P(A) - Q(A)|$$

For continuous distributions:

$$TV(P, Q) = \frac{1}{2} \int |p(x) - q(x)| dx$$

It provides an upper bound on distribution differences, ranges from 0 to 1, and is symmetric.

Limitations include computational challenges and less sensitivity than other metrics.

Selection Criteria

Method selection depends on data type, dimensionality, interpretability needs, computational constraints, and integration requirements. PSI is preferred for interpretability and simplicity; KS is suitable for continuous features and statistical rigor; Chi-square is appropriate for categorical features; Wasserstein and MMD are useful for high-dimensional continuous data.

Combinatorial Approaches

Multiple methods can be combined to improve robustness. Ensemble approaches aggregate results from multiple tests, multi-scale methods apply tests at different granularities, and hierarchical approaches first assess dataset-level drift and then examine specific features.

1.5. Оценка эффективности различных статистических критериев для выявления дрифта

Assessing statistical criteria requires comparing their sensitivity, computational efficiency, interpretability, robustness, and applicability across scenarios. Effectiveness depends on drift type, data characteristics, and operational constraints.

Sensitivity and Statistical Power

Sensitivity is the ability to detect true drift while avoiding false positives. The Kolmogorov-Smirnov test is sensitive to differences in distribution centres and tails, but may be less powerful for tail-heavy or multimodal distributions. Population Stability Index provides good sensitivity for moderate to large shifts, though binning can reduce sensitivity to subtle shifts. Chi-square tests are sensitive for categorical features but can miss patterns in high-cardinality categories. Maximum Mean Discrepancy can be highly sensitive with appropriate kernels, especially for complex, high-dimensional shifts, though kernel choice affects sensitivity. Wasserstein distance is sensitive to shape differences and preserves distribution geometry, making it useful for continuous data.

Type I and Type II Error Rates

Type I errors (false positives) are costly when they trigger unnecessary retraining. The KS test provides explicit significance levels, allowing control over Type I errors via p-value thresholds. PSI lacks formal significance testing, so threshold selection is more subjective. Chi-square tests provide formal significance but can be sensitive to small-expected frequencies. MMD can have higher false positive rates if the kernel is not well tuned. Wasserstein distance lacks standard significance tests, requiring empirical threshold selection. Type II errors (false negatives) allow degraded models to remain in production. The KS test may miss tail differences or high-dimensional drift. PSI may miss subtle shifts if binning is inappropriate. Chi-square may miss shifts in high-cardinality categorical features. MMD and Wasserstein generally have good power for continuous distributions, but computational constraints can limit their use.

Computational Complexity and Scalability

Computational requirements matter for real-time or high-volume systems. PSI is efficient: $O(n)$ for binning and counting, suitable for large datasets. The KS test is efficient: $O(n \log n)$ for sorting and $O(n)$ for computing the statistic, good for medium to large datasets. Chi-square is efficient: $O(k)$ where k is the number of categories, very fast for categorical data. Wasserstein distance can be expensive: $O(n^2)$ for general cases, though faster approximations exist for one-dimensional data. MMD is moderately expensive: $O(n^2)$ for kernel evaluations, though efficient approximations are available. For high-dimensional data, curse of dimensionality affects all methods. PSI requires binning each dimension or dimensionality reduction. KS is typically applied dimension-by-dimension. Chi-square applies per categorical feature. MMD can handle high dimensions with appropriate kernels but at increased cost. Wasserstein distance becomes computationally prohibitive in high dimensions.

Interpretability and Actionability

Interpretability affects adoption and troubleshooting. PSI provides interpretable scores (low/moderate/high) and identifies contributing bins. KS provides a test statistic and p-value with clear statistical meaning but less intuitive feature-level insights. Chi-square identifies problematic categories and provides clear statistical interpretation. MMD gives a single score but limited interpretability for which features drive drift. Wasserstein distance provides interpretability as transport cost but less intuitive for stakeholders.

Robustness to Data Characteristics

Methods differ in robustness to outliers, missing data, and distribution assumptions. PSI is somewhat robust to outliers (depending on binning), handles missing values with appropriate binning, and is non-parametric. The KS test is sensitive to outliers, requires complete data, and is non-parametric. Chi-square is sensitive to sparse categories, requires complete data, and is non-parametric. MMD can be robust with appropriate kernels, can handle missing data through kernel design, and is non-parametric. Wasserstein distance is moderately robust to outliers, sensitive to missing data, and non-parametric.

Threshold Selection and Calibration

Selecting thresholds balances sensitivity and false positives. PSI uses industry-standard thresholds (0.1, 0.25) that are simple but may not generalize. KS uses p-values with standard significance levels (0.05, 0.01), providing formal control but potentially high false positives in large datasets. Chi-square uses p-values, similar to KS. MMD and Wasserstein often require empirical thresholds, which need calibration per use case.

Comparative Performance Studies

Empirical studies show different strengths. For sudden, large shifts, most methods perform well. For gradual drift, MMD and Wasserstein may detect earlier; KS and PSI may lag. For high-dimensional data, MMD and MMD variants often outperform dimension-wise methods. For categorical data, Chi-square is typically most appropriate. For streaming data, PSI and incremental KS variants are more practical.

Recommendations for Method Selection

Select based on data type, interpretability needs, computational constraints, and operational requirements. For continuous features where interpretability matters, PSI is recommended. For continuous features where statistical rigor matters, the KS test is recommended. For categorical features, Chi-square is recommended. For high-dimensional continuous data, MMD is recommended. For continuous data where shape matters, Wasserstein distance is recommended.

Hybrid and Ensemble Approaches

Combining methods can improve robustness. Ensemble approaches aggregate multiple methods to reduce false positives and negatives. Hierarchical approaches use dataset-level tests first, then feature-level tests. Adaptive approaches adjust method selection based on data characteristics.

Practical Considerations

Consider operational constraints, maintenance requirements, integration complexity, and stakeholder communication. PSI offers simplicity, interpretability, and broad acceptance in finance. KS provides statistical rigor and standard software support. Chi-square is standard for categorical analysis. MMD offers flexibility for complex data but requires more tuning. Wasserstein distance provides intuitive interpretation but requires more computational resources.

1.6. Современные практики автоматизации процессов обновления моделей

Automating model updates requires systematic approaches that detect when updates are needed, collect appropriate data, retrain models, evaluate new models, and deploy them safely. Practices vary across organizations, but common patterns include scheduled retraining, performance-triggered retraining, and drift-triggered retraining.

Retraining Trigger Mechanisms

Triggers determine when retraining occurs. Scheduled retraining runs on fixed intervals regardless of performance or drift, ensuring regular updates but potentially wasting resources. Performance-triggered retraining initiates when metrics degrade below thresholds, directly addressing performance issues but requiring labels. Drift-triggered retraining responds to distribution shifts, enabling proactive updates without labels, though it may not always correlate with performance issues. Hybrid approaches combine multiple triggers, such as scheduled checks with drift-based acceleration or performance monitoring with drift validation. This balances timeliness and resource efficiency.

Data Collection and Management Strategies

Data collection strategies vary. Incremental collection appends new data to existing datasets, suitable for stable environments. Sliding window approaches maintain a fixed-size window of recent data, focusing on recent patterns. Stratified sampling ensures balanced class distributions. Active learning prioritizes informative samples to reduce labeling costs. Data quality validation checks for missing values, outliers, and schema consistency before retraining. Data versioning tracks training data versions and links models to data for reproducibility.

Retraining Strategies

Retraining strategies differ in scope and efficiency. Full retraining trains from scratch on the complete dataset, ensuring the model sees all data but being computationally expensive. Incremental learning updates the model with new data, efficient but potentially losing information from older data. Transfer learning adapts a pre-trained model to new data, efficient and often effective. Ensemble methods combine multiple models, robust but more complex.

Model Evaluation and Comparison

Evaluation compares new models with production models. Hold-out validation uses held-out data for unbiased evaluation. Cross-validation provides robust estimates but is computationally expensive. Temporal validation uses recent data to assess performance on future data, realistic for time-sensitive applications. Comparison metrics include accuracy, precision, recall, F1-score, AUC-ROC for classification, and MSE, MAE, R^2 for regression. Business metrics may include revenue impact, user engagement, and cost savings. Statistical significance tests determine if improvements are meaningful.

Deployment Strategies

Safe deployment minimizes risk. Shadow deployment runs new models alongside production models without affecting users, enabling validation before full deployment. Canary deployment gradually increases traffic to new models, allowing rollback if issues arise. A/B testing compares models on user subsets, providing statistical validation. Blue-green deployment maintains two environments for instant rollback. Rollback mechanisms are critical. Version control maintains previous model versions. Performance monitoring tracks metrics post-deployment. Automated rollback triggers revert to previous versions if metrics degrade.

Orchestration and Workflow Management

Orchestration coordinates retraining workflows. Workflow definition specifies tasks and dependencies. Scheduling manages when workflows run. Dependency management ensures correct task ordering. Error handling includes retries, logging, and alerting. Resource management allocates computational resources efficiently. Popular orchestration tools include Apache Airflow (DAG-based), Prefect (Python-native), Kubeflow Pipelines (Kubernetes-integrated), and MLflow Pipelines (ML-specific).

Model Registry and Versioning

Model registries track model versions, metadata, and lineage. Versioning assigns unique identifiers and tracks changes. Metadata includes training parameters, performance metrics, and data versions. Lineage tracks relationships between models, data, and code. Status management tracks development, staging, production, and archived states.

Continuous Integration and Continuous Deployment (CI/CD)

CI/CD practices apply to ML systems. Continuous integration validates code changes through automated testing. Continuous deployment automates deployment to production. MLOps pipelines integrate model training, evaluation, and deployment. ML-specific considerations include model testing (validation on hold-out data), data testing (distribution checks), and performance testing (metric thresholds).

Monitoring and Observability

Post-deployment monitoring tracks performance, data distributions, and system health. Performance monitoring tracks accuracy, latency, and throughput. Data monitoring tracks input distributions and quality. System monitoring tracks infrastructure health. Alerting notifies stakeholders of issues.

Challenges and Best Practices

Challenges include data availability, computational resources, model validation, deployment risk, and maintenance complexity. Best practices include automated testing, gradual rollouts, comprehensive monitoring, documentation, and stakeholder communication.

Emerging Trends

Trends include automated hyperparameter tuning, neural architecture search, online learning for streaming data, federated learning for distributed data, and explainable AI for model interpretability.

1.7. Характеристика экспериментальных данных и источников их получения

Experimental validation requires datasets that exhibit realistic drift patterns and span multiple domains. This section describes the data characteristics, sources, and preparation strategies used in this research.

Data Requirements for Drift Detection Validation

Effective validation needs data with known baseline distributions, observable drift patterns, sufficient volume, appropriate dimensionality, and realistic feature types. Datasets should allow controlled drift introduction for method validation and include natural drift for real-world assessment.

Synthetic Data Generation

Synthetic data enables controlled experiments. Techniques include distribution shifting by modifying parameters, gradual drift through incremental parameter changes, sudden drift via step changes, and recurring drift with periodic patterns. Advantages include controlled conditions, reproducibility, and scalability. Limitations include potential lack of realism and simplified assumptions.

Real-World Datasets with Natural Drift

Real-world datasets provide realistic validation. Time-series datasets, such as financial market data, IoT sensor data, and web traffic data, naturally exhibit temporal shifts. Cross-domain datasets, such as fraud detection, recommendation systems, and customer behavior data, show concept and data drift. Public datasets from UCI Machine Learning Repository, Kaggle, and research publications offer diverse scenarios.

Dataset Characteristics

Datasets vary in size, dimensionality, feature types, and temporal characteristics. Small datasets (thousands of samples) suit rapid experimentation. Medium datasets (hundreds of thousands) balance realism and computational cost. Large datasets (millions) test scalability. Dimensionality ranges from low (tens) to high (hundreds or thousands), with high-dimensional data presenting unique challenges. Feature types include continuous (real-valued), categorical (nominal/ordinal), mixed (continuous and categorical), and text/image features requiring specialized handling. Temporal characteristics include static baselines, gradual shifts, sudden shifts, recurring patterns, and multiple shift types.

Specific Datasets for Experimental Validation

Several datasets will be used. Financial transaction datasets include credit card fraud detection with temporal patterns and transaction amount distributions. E-commerce datasets include customer behavior with seasonal patterns and preference shifts. IoT sensor datasets include sensor measurements with calibration drift and environmental changes.

Data Preprocessing and Preparation

Preprocessing standardizes data for drift detection. Feature engineering creates relevant features. Normalization standardizes scales. Handling missing data uses imputation or appropriate binning. Outlier treatment uses robust methods or outlier bins. Categorical encoding uses one-hot encoding or frequency encoding. Baseline establishment defines training data distributions. Statistics computation calculates means, variances, and distributions. Baseline storage saves baseline statistics for comparison. Production simulation creates production-like scenarios. Temporal splitting separates training and production periods. Drift introduction (for synthetic data) adds controlled shifts. Data streaming simulates continuous data arrival.

Data Quality Assurance

Quality assurance ensures reliable experiments. Schema validation checks feature consistency. Range validation checks value ranges. Completeness checks verify data availability. Distribution validation ensures expected distributions.

Ethical and Privacy Considerations

When using real-world data, address privacy, consent, bias, and fairness. Anonymize or use synthetic data when possible. Obtain appropriate consent. Assess and mitigate bias. Ensure fair representation.

Data Versioning and Reproducibility

Reproducibility requires data versioning. Version control tracks data versions. Metadata documentation records preprocessing steps. Hash verification ensures data integrity. Access control manages data access.

Computational Resources and Storage

Consider storage and computational requirements. Storage needs depend on dataset size and versioning strategy. Computational resources depend on data processing and detection method complexity. Cloud resources may be needed for large-scale experiments.

1.8. Предлагаемый подход к построению единой платформы мониторинга и адаптации моделей

This section presents the conceptual architecture of an integrated MLOps system that unifies drift detection, workflow orchestration, automated retraining, and model management in a single platform.

Design Philosophy and Core Principles

The system is built on several principles. Integration unifies detection, orchestration, retraining, and management. Automation minimizes manual intervention. Modularity allows independent component development and testing. Extensibility supports new detection methods and strategies. Observability provides visibility into system behavior. Reliability ensures robust operation with error handling and recovery.

Architectural Overview

The system consists of interconnected modules. The Drift Detection Module monitors production data and identifies distribution shifts. The Orchestration Module manages workflows and task dependencies. The Model Management Module handles versioning, registry, and lifecycle. The Retraining Module executes automated retraining workflows. The Data Management Module handles versioning and storage. The API Layer provides programmatic access. The Monitoring Dashboard provides visual interfaces.

Drift Detection Module Architecture

The Drift Detection Module continuously monitors production data. The Data Ingestion Component collects production data in batches or streams. The Baseline Manager stores and retrieves baseline statistics. The Statistical Engine computes PSI, KS, Chi-square, and other metrics. The Aggregation Component combines feature-level results into dataset-level assessments. The Decision Engine determines drift severity and triggers. The Results Storage component persists detection results. The module supports multiple detection methods, configurable thresholds, batch and streaming modes, and detailed reporting.

Orchestration Module Architecture

The Orchestration Module coordinates system workflows. The Scheduler manages task scheduling. The Workflow Engine executes workflows with dependency resolution. The Task Executor runs individual tasks. The State Manager tracks workflow and task states. The Error Handler manages failures and retries. Workflow types include periodic drift checks, drift-triggered retraining, scheduled retraining, and manual interventions. The module supports dependency management, parallel execution, error handling, and state persistence.

Model Management Module Architecture

The Model Management Module handles the complete model lifecycle. The Model Registry stores metadata, versions, and relationships. The Model Storage manages serialized model files. The Versioning System tracks versions and history. The Deployment Manager handles deployment. The Lineage Tracker maintains relationships between models, data, and code. The module supports version control, metadata tracking, status management, and deployment automation.

Retraining Module Architecture

The Retraining Module executes automated retraining workflows. The Data Collector gathers training data. The Data Merger combines datasets. The Preprocessing Component applies preprocessing

pipelines. The Training Component trains models. The Evaluation Component assesses performance. The Comparison Component compares new and existing models. The Deployment Component handles deployment. The module supports multiple strategies (full, incremental, transfer learning), automated evaluation, performance comparison, and conditional deployment.

Data Management Module Architecture

The Data Management Module handles data versioning and storage. The Version Control System tracks data versions. The Storage Manager manages data files. The Schema Registry tracks data schemas. The Integrity Checker validates data integrity. The module supports versioning, integrity checking, schema management, and efficient storage.

API Layer Architecture

The API Layer provides programmatic access. REST endpoints support model registration, drift checking, retraining triggers, workflow management, and result retrieval. Authentication and authorization ensure secure access. API versioning maintains compatibility.

Monitoring Dashboard Architecture

The Monitoring Dashboard provides visual interfaces. The Model Overview shows registered models and statuses. The Drift Monitor displays detection results and trends. The Workflow Monitor shows workflow execution and status. The Performance Monitor tracks model performance. The System Health shows overall system status. The dashboard supports real-time updates, historical analysis, interactive visualizations, and alerting.

Data Flow and Integration

Data flows through the system in coordinated paths. Initial model registration establishes baselines. Continuous monitoring collects production data and performs detection. Drift-triggered workflows initiate retraining when needed. Retraining workflows execute model updates. Deployment workflows promote models to production. Feedback loops incorporate production feedback.

Configuration and Customization

The system is highly configurable. Detection methods and thresholds are configurable. Retraining strategies and parameters are adjustable. Workflow schedules and dependencies are customizable. Alerting rules and notifications are configurable.

Scalability and Performance Considerations

The system is designed for scalability. Horizontal scaling supports increased load. Efficient algorithms minimize computational overhead. Caching reduces redundant computations. Asynchronous processing improves responsiveness. Resource management optimizes resource usage.

Security and Reliability

Security and reliability are critical. Authentication and authorization control access. Data encryption protects sensitive data. Audit logging tracks system activities. Error handling and recovery ensure robustness. Backup and recovery protect against data loss.

Comparison with Existing Solutions

The proposed system differs from existing solutions by providing unified integration, custom-built components, lightweight architecture, complete automation, and open-source availability. It eliminates the need for multiple tool integrations, provides fine-grained control, reduces complexity, and enables comprehensive automation.

Implementation Strategy

Implementation proceeds in phases. Phase 1 focuses on core drift detection. Phase 2 adds orchestration and workflow management. Phase 3 implements model management and registry. Phase 4 adds retraining automation. Phase 5 develops APIs and dashboard. Phase 6 includes testing and validation.

Expected Contributions

The system contributes by demonstrating unified MLOps architecture, providing open-source implementation, advancing drift detection methodologies, and establishing best practices for automated retraining.

Bibliography

